Name :- Akshat Chauhan

BTech Cse "C"

Cu24250136

Roll no. = 05

Experiment No. 3

Experiment Name

Implementation and Analysis of Spatial Filtering Techniques

using Low-Pass and High-Pass Filters

order to identify relationships between variables, validate assumptions, and support data-driven decision-making.

To implement and analyze spatial domain filtering techniques

using low-pass and high-pass filters for image smoothing,

using Python and OpenCV.

Step 1: Import Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from tkinter import Tk, filedialog
```

Step 2: Upload / Select Image from Computer

```
from google.colab import files
import io

print("Please upload your image file.")
uploaded = files.upload()

if not uploaded:
    print("No image selected.")
    exit()

# Assuming only one file is uploaded, get its name
image_name = next(iter(uploaded))
image_data = uploaded[image_name]

# Read image data into a numpy array
nparr = np.frombuffer(image_data, np.uint8)
image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

if image is None:
    print("Error: Unable to load the selected image. Please ensure it is a valid image file.")
    exit()

# Convert image to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)

# Convert image to grayscale
gray = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2GRAY
)

print("Image uploaded successfully.")
print("Selected Image:", image_name)
print("Image Shape:", image.shape)
```

```
Please upload your image file.
Choose Files No file chosen
Uploadwidgetisonlyavailablewhenthecellhasbeenexecuted in the current browser session. Please rerun this cell to enable.
Saving Screenshot 2025-02-26 152118.png to Screenshot 2025-02-26 152118.png
Image uploaded successfully.
Selected Image: Screenshot 2025-02-26 152118.png
Image Shape: (1077, 1919, 3)
```

Step 3: Display Original Image

```
plt.figure(figsize=(6, 5))

plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.show()

print("Original image displayed successfully.")
```




```
Original image displayed successfully.
```

Step 4: Apply Gaussian Blur

```
gaussian_filter = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)
```

```
print("Gaussian Blur applied successfully.")
```

```
Gaussian Blur applied successfully.
```

Step 5: Apply Median Filtering

```
median_filter = cv2.medianBlur(
    gray,
    5
)

print("Median Filtering applied successfully.")
```

```
Median Filtering applied successfully.
```

Step 6: Apply Average / Mean Filtering

```
average_filter = cv2.blur(
    gray,
    (5, 5)
)

print("Average Filtering applied successfully.")
```

```
Average Filtering applied successfully.
```

Step 7: Apply Laplacian Filtering

```
laplacian = cv2.Laplacian(
    gray,
    cv2.CV_64F
)

laplacian = cv2.convertScaleAbs(
    laplacian
)

print("Laplacian Filtering applied successfully.")
```

```
Laplacian Filtering applied successfully.
```

Step 8: Apply Sobel Edge Detection

```
sobel_x = cv2.Sobel(
    gray,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

sobel_x = cv2.convertScaleAbs(
    sobel_x
)

# Sobel Y - Detects horizontal edges
sobel_y = cv2.Sobel(
    gray,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

sobel_y = cv2.convertScaleAbs(
    sobel_y
)

print("Sobel X Edge Detection applied successfully.")
print("Sobel Y Edge Detection applied successfully.")
```

```
Sobel X Edge Detection applied successfully.
Sobel Y Edge Detection applied successfully.
```

Step 9: Display All Filtered Images

```
plt.figure(figsize=(12, 10))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gaussian_filter, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

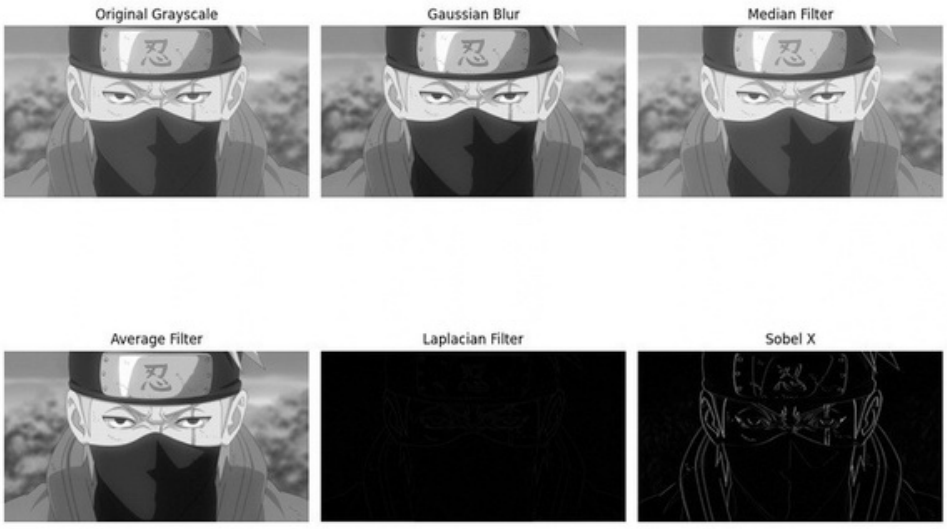
plt.subplot(2, 3, 3)
plt.imshow(median_filter, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(average_filter, cmap="gray")
plt.title("Average Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 10: Display Sobel Y Separately

```
plt.figure(figsize=(6, 5))

plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.show()

print("All filtered images displayed successfully.")
```




```
All filtered images displayed successfully.
```

Step 11: Final Output

```
print("\n========== SPATIAL FILTERING ==========")

print("Original Image Shape:", image.shape)

print("Gaussian Blur: Completed")
print("Median Filtering: Completed")
print("Average Filtering: Completed")
print("Laplacian Filtering: Completed")
print("Sobel X Edge Detection: Completed")
print("Sobel Y Edge Detection: Completed")



print("\nObservations:")

print("Gaussian Filter smooths the image and reduces noise.")
print("Median Filter is effective for removing salt-and-pepper noise.")
print("Average Filter performs simple image smoothing.")
print("Laplacian Filter enhances edges and fine details.")
print("Sobel X detects vertical edges.")
print("Sobel Y detects horizontal edges.")

print("\nExperiment Completed Successfully.")
```

```
========== SPATIAL FILTERING ==========
Original Image Shape: (1077, 1919, 3)
Gaussian Blur: Completed
Median Filtering: Completed
Average Filtering: Completed
Laplacian Filtering: Completed
Sobel X Edge Detection: Completed
Sobel Y Edge Detection: Completed



Observations:
Gaussian Filter smooths the image and reduces noise.
Median Filter is effective for removing salt-and-pepper noise.
Average Filter performs simple image smoothing.
Laplacian Filter enhances edges and fine details.
Sobel X detects vertical edges.
Sobel Y detects horizontal edges.



Experiment Completed Successfully.
```

EXPERIMENT 3 - ANSWERS TO QUESTIONS

QUESTION 1: What is spatial filtering? How is it used in digital image processing?

Spatial filtering is an image processing technique in which pixel values are modified based on the values of neighboring pixels.

A small matrix called a kernel or filter mask is moved over the image and mathematical operations are performed on the pixels within the neighborhood.

Spatial filtering is used for:

Noise reduction Image smoothing Edge detection Image sharpening Feature enhancement In this experiment, Gaussian, Median, Average, Laplacian and Sobel filters are used for spatial filtering.

==========

QUESTION 2: Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Low-Pass Filters:

Low-Pass Filters allow slow variations and low-frequency information to pass while reducing high-frequency components.

They are mainly used for:

Image smoothing Noise reduction Blur Examples:

Gaussian Filter Average Filter Median Filter

High-Pass Filters:

High-Pass Filters emphasize high-frequency components such as edges, fine details and sudden intensity changes.

They are mainly used for:

Edge detection Image sharpening Feature enhancement Examples:

Laplacian Filter Sobel Filter

Main Difference:

Low-Pass Filter -> Smooths image and reduces noise. High-Pass Filter -> Enhances edges and fine details.

==========

QUESTION 3: Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Average Filter:

The Average Filter replaces each pixel with the average value of its neighboring pixels.

It provides simple smoothing but can blur edges.

Application: General image smoothing and noise reduction.

Gaussian Filter:

The Gaussian Filter uses a weighted average where pixels closer to the center have greater importance.

It provides smoother and more natural results compared to the simple Average Filter.

Application: Noise reduction and image smoothing while preserving important structures better than simple averaging.

Median Filter:

The Median Filter replaces a pixel with the median value of its neighboring pixels.

It is particularly effective against impulse or salt-and-pepper noise.

Application: Removal of salt-and-pepper noise while preserving edges.

Comparison:

Average -> Simple averaging, can blur edges. Gaussian -> Weighted averaging, smoother results. Median -> Uses median value, excellent for impulse noise.

==========

QUESTION 4: Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Salt-and-pepper noise consists of random bright and dark pixels appearing in an image.

The Median Filter replaces each pixel with the median value of its neighborhood.

Since the median is less affected by extreme values than the average, noisy pixels are effectively removed.

At the same time, edges are generally preserved better than with an Average Filter.

Therefore, Median Filtering is particularly effective for removing salt-and-pepper noise.

==========

QUESTION 5: Explain the role of convolution kernels in spatial filtering.

A convolution kernel is a small matrix containing numerical values that defines how neighboring pixels should be combined.

The kernel moves across the image and performs a mathematical operation with the pixels in each local neighborhood.

Different kernels produce different effects.

Examples:

Gaussian Kernel -> Smoothing Average Kernel -> Blurring Sobel Kernel -> Edge Detection Laplacian Kernel -> Edge Enhancement

Therefore, convolution kernels are fundamental components of spatial filtering.

==========

QUESTION 6: What is the purpose of the Sobel and Laplacian operators in edge detection?

Sobel Operator:

The Sobel operator calculates image intensity gradients.

Sobel X detects changes primarily in the horizontal direction and therefore highlights vertical edges.

Sobel Y detects changes primarily in the vertical direction and therefore highlights horizontal edges.

Laplacian Operator:

The Laplacian operator is a second-order derivative operator.

It detects regions where image intensity changes rapidly.

It can enhance edges and fine details in an image.

Therefore:

Sobel -> Gradient-based edge detection. Laplacian -> Second-order edge enhancement.

==========

QUESTION 7: Why are filtering operations considered an essential preprocessing step in computer vision?

Filtering is an important preprocessing step because it improves the quality of an image before further processing.

Filtering can:

Remove unwanted noise. Smooth image variations. Enhance important edges. Improve feature visibility. Reduce unwanted image details. Improve the input provided to computer vision algorithms. Better image quality can help subsequent tasks such as object detection, segmentation, recognition and feature extraction.

==========

QUESTION 8: Discuss the trade-off between image smoothing and edge preservation during filtering.

Image smoothing reduces noise and unwanted variations.

However, excessive smoothing can also remove important edges and fine image details.

Strong filtering: -> More noise reduction -> More loss of edges and details

Weak filtering: -> Better edge preservation -> More noise may remain

Therefore, an appropriate filter and kernel size must be selected according to the application.

Median and Gaussian Filters can provide a useful balance between noise reduction and preservation of important structures.

==========

QUESTION 9: Mention four real-world applications where spatial filtering techniques are widely used.

Four real-world applications are:

Medical Imaging: Filtering is used to reduce noise and improve the quality of medical images such as X-rays, CT scans and MRI images.

Surveillance Systems: Filtering can improve image quality and enhance important features in surveillance footage.

Remote Sensing: Spatial filters can be used to enhance satellite and aerial images.

Autonomous Systems: Filtering can improve camera images before object detection, recognition and other computer vision operations.

These applications demonstrate the importance of spatial filtering in practical computer vision systems.

==========

QUESTION 10: Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Spatial Domain Filtering:

Spatial filtering operates directly on image pixels.

It generally uses convolution kernels or neighborhood operations.

Examples:

Gaussian Filter Median Filter Average Filter Sobel Filter Laplacian Filter

It is commonly used for:

Smoothing Noise removal Edge detection Sharpening Frequency Domain Filtering:

Frequency domain filtering first converts an image from the spatial domain into the frequency domain using Fourier Transform.

The frequency components are then modified using filters.

It is commonly used for:

Noise removal Image restoration Frequency-based enhancement Selective manipulation of image frequencies Main Difference:

Spatial Domain: Directly modifies pixel values using local neighborhoods.

Frequency Domain: Modifies frequency components after Fourier Transform.

Spatial filtering is generally simpler for local image operations, while frequency domain filtering is useful when selective control over frequency components is required.